



Onboard 3D Drone Detection and Tracking



BOLD
BUSINESS

Image Source: https://www.boldbusiness.com/wp-content/uploads/2019/06/Vorpal-Anti-drone_featured.jpg

Project Presentation for AE640: Autonomous Navigation (Fall 2020)

By: Apurva Nandan and Nitik Jain

Advisor: Dr. Mangal Kothari

Motivation

Object detection is one of the most important and challenging branches of computer vision, which has been widely applied in life, such as monitoring security, autonomous driving and so on, with the purpose of locating instances of semantic objects of a certain class.

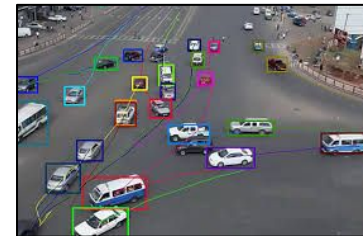
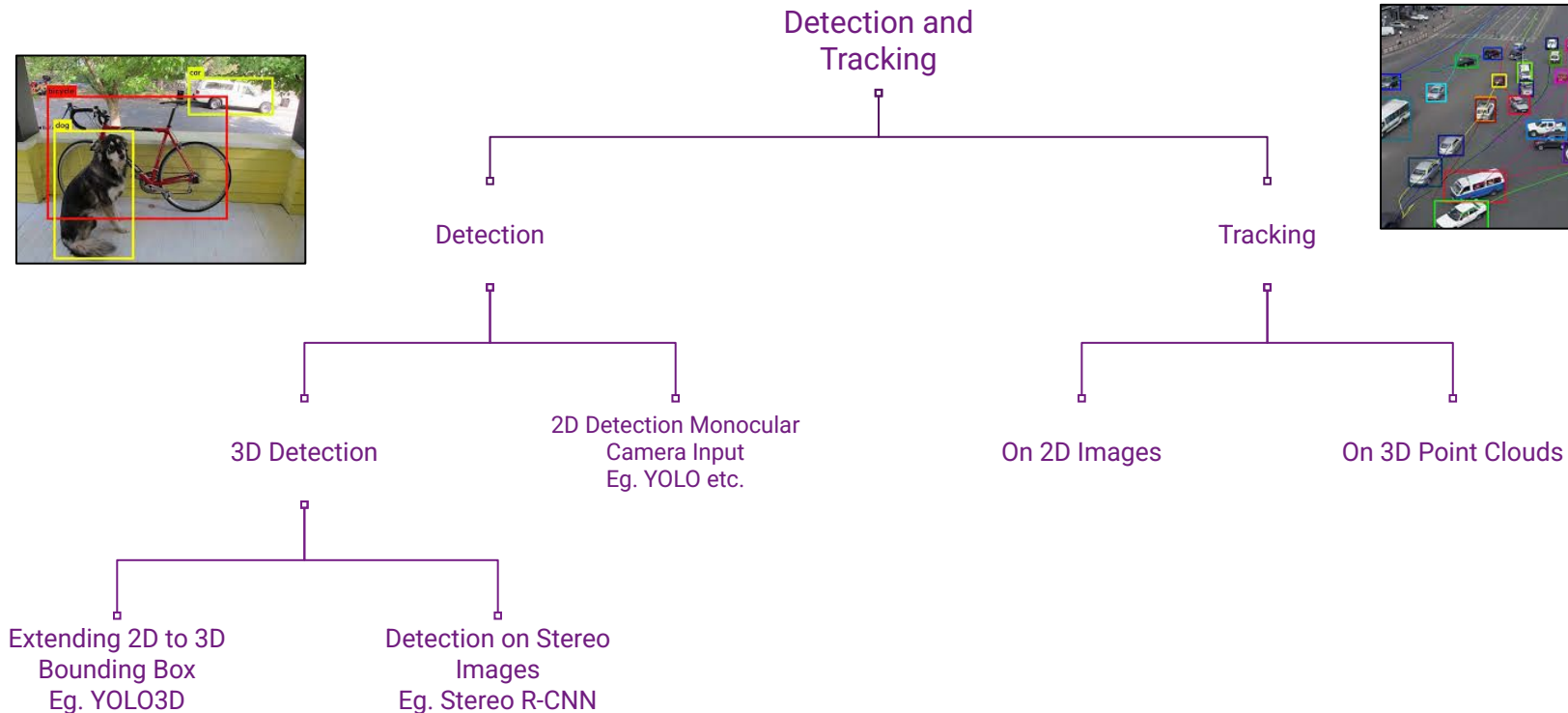
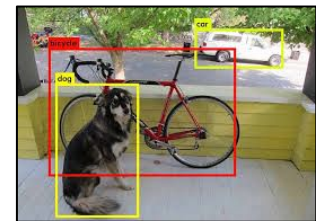
Consequently, with the development of computationally heavier algorithms, the need to apply them in real-time applications arose.

Since, real-time applications needed pipelines to be lightweight as well as quick, various modifications to SOTA algorithms was essential to strip down unnecessary parts.

Here, the project aims to develop pipeline for On Board Drone Detection from another Drone

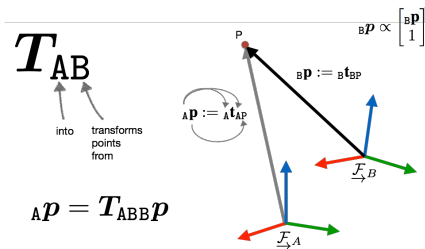
Onboard 3D Drone Detection and Tracking

Since the Problem Statement is pretty exhaustive, it was broken down into the following deliverables



Problem Visualization

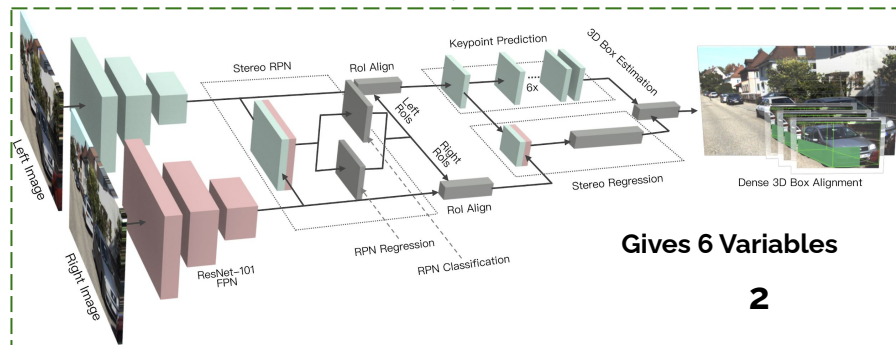
For efficient tracking, we need to find pose wrt us



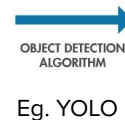
Need 6 Variables

Detection on RGB-D images

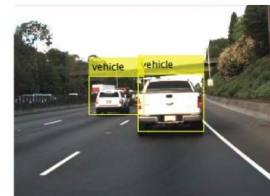
YOLO 3D or Stereo RCNN



Approach 1: Detection on RGB



Eg. YOLO



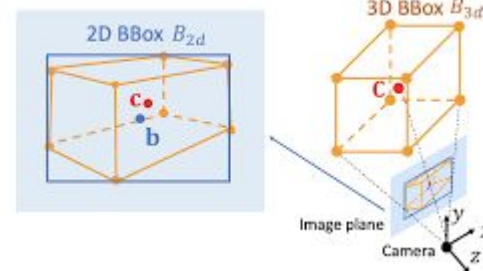
Visual Input feed as RGB or RGB-D Data

Detection on RGB gives 2 variables

1

Assume 2 variables pitch and roll to be zero,

Extend and estimate other 2 variables with regression

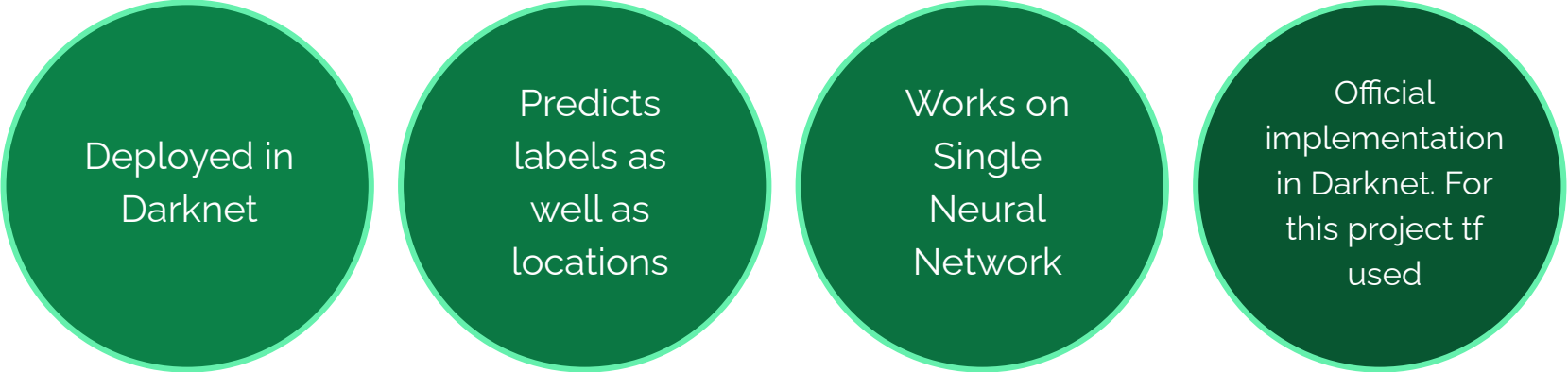


Problem Statement

Designing an end-to-end pipeline for 3D object detection and tracking, to be implemented on a UAV, keeping all processing lightweight and onboard.

- Task 1: Object Detection from Monocular Images (RGB)
 - Using 2D SOTA Algorithm: YOLOv3 [\[Link\]](#)
- Task 2: 3D Object Detection:
 - Extending 2D to 3D
 - 3D Bounding Box Estimation Using Deep Learning and Geometry [\[Link\]](#)
 - YOLO 3D on RGB-D [\[Link\]](#)
 - Stereo-RCNN [\[Link\]](#)
- Task 3: Depth Map based 3D Drone Detection
 - 2D drone detection [\[Link\]](#)
 - Filtering selected RoI in the point cloud based on 2D bounding box
 - Centroid computation
 - 3D bounding box generation

Task 1: About YOLOv3



Deployed in
Darknet

Predicts
labels as
well as
locations

Works on
Single
Neural
Network

Official
implementation
in Darknet. For
this project tf
used

Algorithm: Task 1 (YOLOv3 on RGB Images)

- **Dependencies:**

To build the YOLO we will require : 1. Tensorflow (GPU version preferred for Deep Learning) 2. NumPy (for Numeric Computation) 3. Pillow/PIL (for Image Processing) 4. IPython (for displaying images in Jupyter Notebook) 5. Glob (for finding pathname of all the files in a folder)

- **Feature Extraction:** For feature extraction, Yolo uses Darknet-53 neural net pre-trained on ImageNet. Same as ResNet, Darknet-53 has shortcut (residual) connections, which help information from earlier layers flow further. We omit the last 3 layers (Avgpool, Connected, and Softmax) since we only need the features.
- **Non-max suppression:** The model is going to produce a lot of boxes, so we need a way to discard the boxes with low confidence scores. Also, to avoid having multiple boxes for one object, we will discard the boxes with high overlap as well using non-max suppression for each class.

Architecture: Task 1 (YOLOv3 on RGB Images)

- **Model Hyperparameters:**

```
In [2]: _BATCH_NORM_DECAY = 0.9  
        _BATCH_NORM_EPSILON = 1e-05  
        _LEAKY_RELU = 0.1  
        _ANCHORS = [(10, 13), (16, 30), (33, 23),  
                     (30, 61), (62, 45), (59, 119),  
                     (116, 90), (156, 198), (373, 326)]  
        _MODEL_SIZE = (416, 416)
```

- **Pre-Processing: Batch Normalization:** It is a preprocessing step of features extracted from previous layers, before feeding it to the next layers of the network. We normalize the input layer by adjusting and scaling the activations
- **Activation Function:** Leaky ReLU
- **Anchors:** Anchors are sort of bounding box priors, that were calculated on the COCO (Microsoft) dataset using k-means clustering. We are going to predict the width and height of the box as offsets from cluster centroids.
- **Batch norm and fixed padding:** Standard as in ResNet on ImageNet

Model: Task 1 (YOLOv3 on RGB Images)

- **Pseudo Algo:**

Args:

*n_classes: Number of class labels.
model_size: The input size of the model.
max_output_size: Max number of boxes to be selected for each class.
iou_threshold: Threshold for the IOU.
confidence_threshold: Threshold for the confidence score.
data_format: The input format.*

Returns:

None.

- **Weights:** official weights converted to tf format (due to unavailability of hardware)
- **Layers:**
 - Convolution layers
 - Detection layers (3 in total)
 - Upsample layerFinally Non-Max Supression
- **Utility Functions:**
 - def load_images(img_names, model_size): (Loads images in a 4D array)
 - def load_class_names(file_name): (Returns a list of class names read from `file\_name`)
 - def draw_boxes(img_names, boxes_dicts, class_names, model_size): (Draws detected boxes)

Results: Task 1 (YOLOv3 on RGB Images)



Due to lack of test data (or any dataset), f-score could not be calculated

Task 2 3D Bounding Box Estimation Using Deep Learning and Geometry

- Implementation of this paper: <https://arxiv.org/abs/1612.00496> (CVPR 2017)
- In contrast to current techniques that only regress the 3D orientation of an object, this method first regresses relatively stable 3D object properties using a deep convolutional neural network and then combines these estimates with geometric constraints provided by a 2D object bounding box to produce a complete 3D bounding box.
- Run on [KITTI Dataset](#)

Assumptions:

- The 2D bounding box fits very tightly around the object
- The object has 0 pitch and 0 roll (valid for cars on the road)

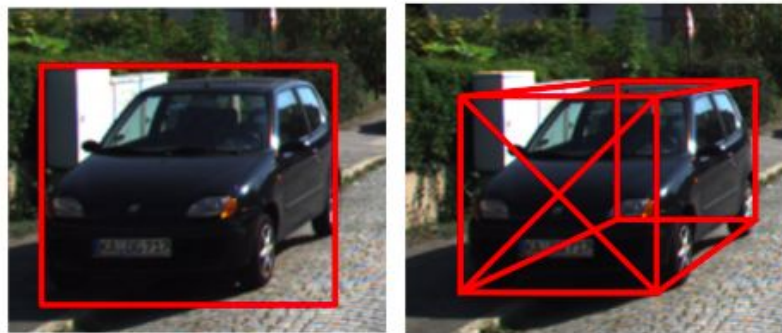
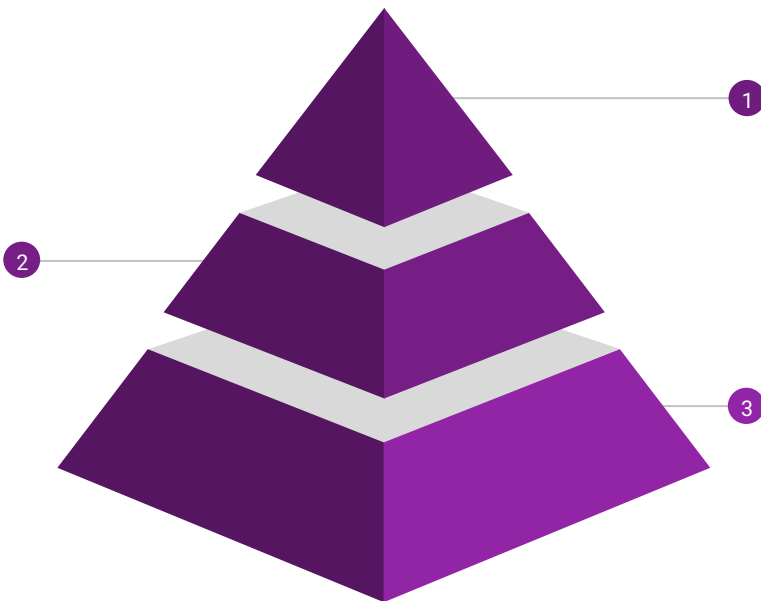


Figure 1. Our method takes the 2D detection bounding box and estimates a 3D bounding box.

Task 2 3D Bounding Box Estimation: How it Works

NeuralNet 2

Using the orientation, dimension, and 2D bounding box, the 3D location is calculated, and then back projected onto the image.



YOLOv3

- 1 2D bounding box and object class

Overall Model

The PyTorch neural net takes in images of size 224x224 and predicts the orientation and relative dimension of that object to the class average



Task 2 3D Bounding Box Estimation: Results

2D
Bounding
Box from
Yolo



Tightly fit 3D
Bounding
Box from
Regression

Task 2 3D Bounding Box Estimation: Results and Limitations



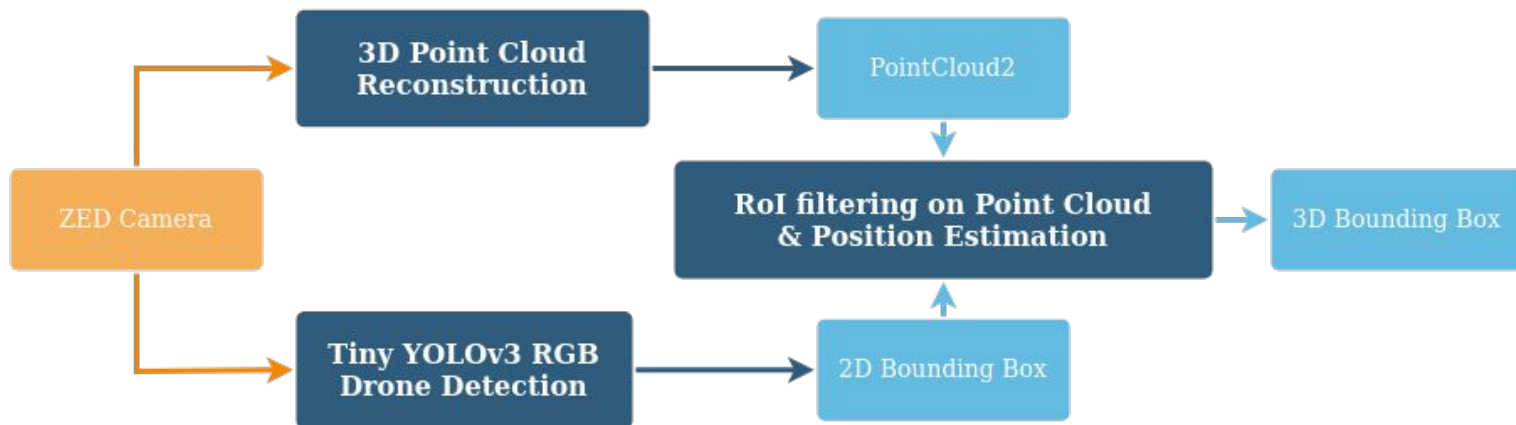
Limitations:

- At the moment, it takes approximately 0.4s per frame, depending on the number of objects detected
- Some type of Pose visualization can't be incorporated, would be looked upon in future work

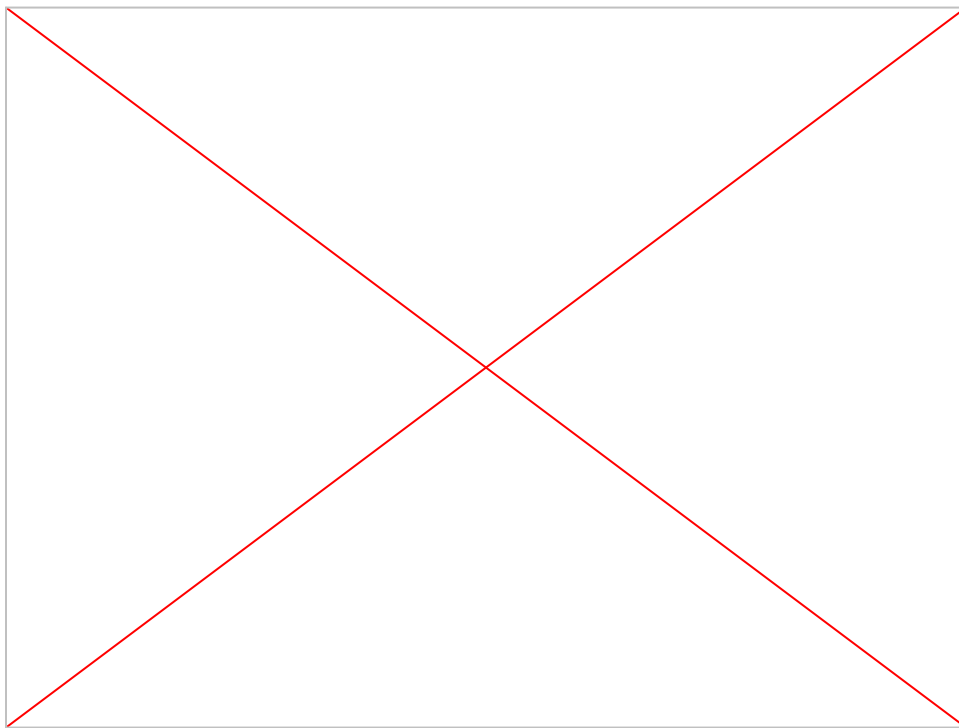
Task 3: Depth Map based 3D Drone Detection

- A flying object is typically isolated in 3D space, with no contact with other objects having the same depth.
- Any drone in a depth map appears as a blob which contrasts with the background.
- This generates a discontinuity in the depth map, which can be used as a distinct visual feature for creating bounding boxes.
- 2D drone detection aids in cropping the depth map with most probable blobs in space.
- By filtering the RoI accurately and applying clustering algorithms, we can fit 3D bounding box on the drones.

Algorithm



2D Drone Detection



Darknet based Tiny YOLOv3 trained over 50K images of drone

3D position estimation using PCL library

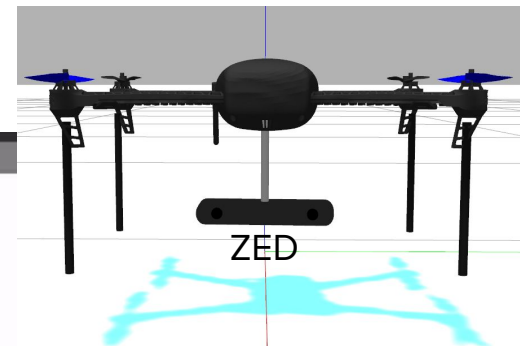
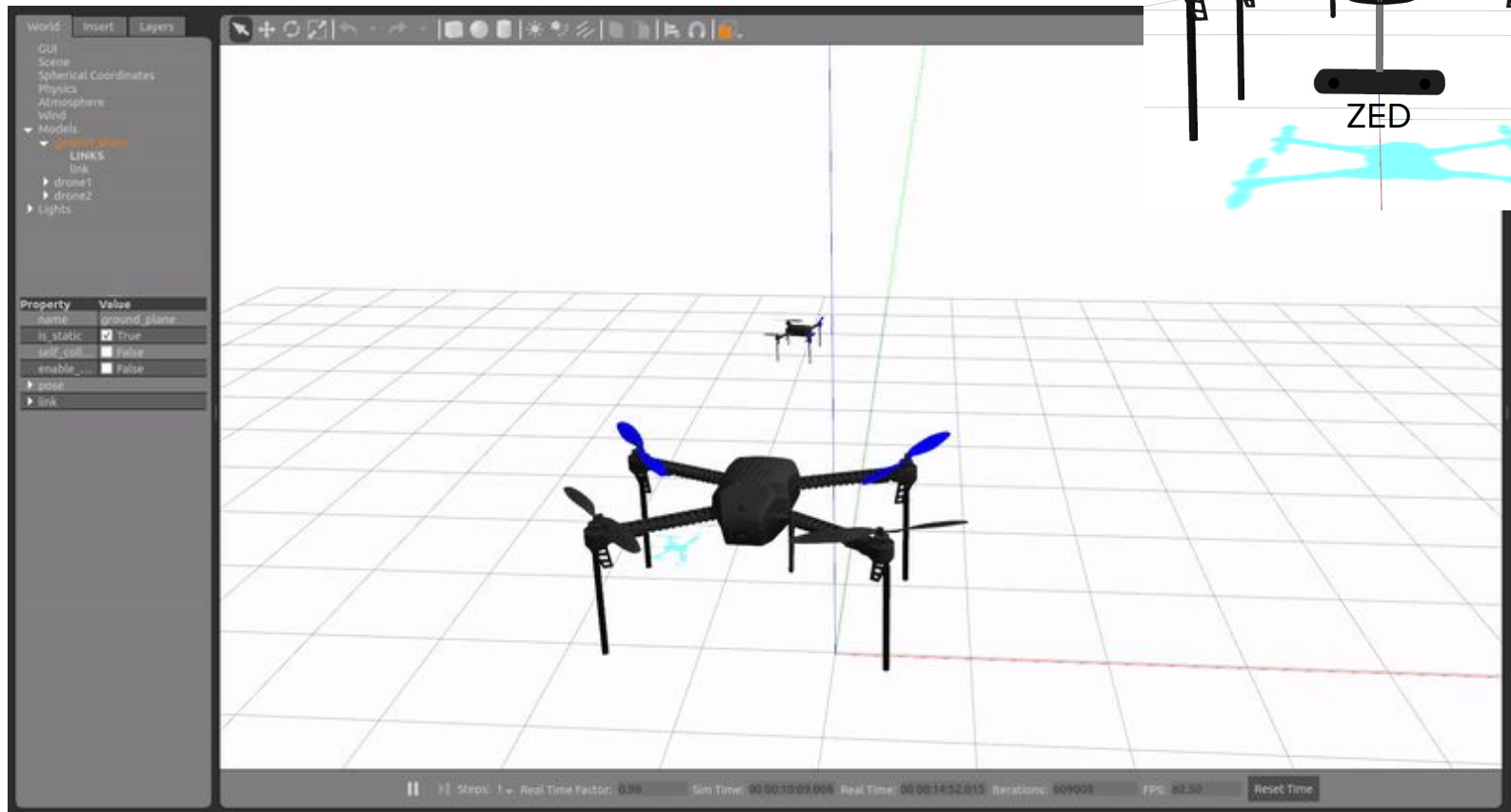
```
zed.SetROI(px1,px2,py1,py2); \\ Trimming the pointcloud
roi = zed.GetCloud();        \\ with the 2D bounding box projection
sor.setInputCloud(roi);
sor.setMeanK(50);            \\ Number of Points for Mean Distance
sor.setStddevMulThresh(1.0); \\ SD multiplier threshold ( $\mu \pm \sigma \cdot std\_mul$ )
sor.filter(*filtered);       \\ Statistical Outlier Removal
if (!filtered->empty())
    pcl::computeCentroid(*filtered,centroid); \\ Centroid Estimation
```

Simulation stack

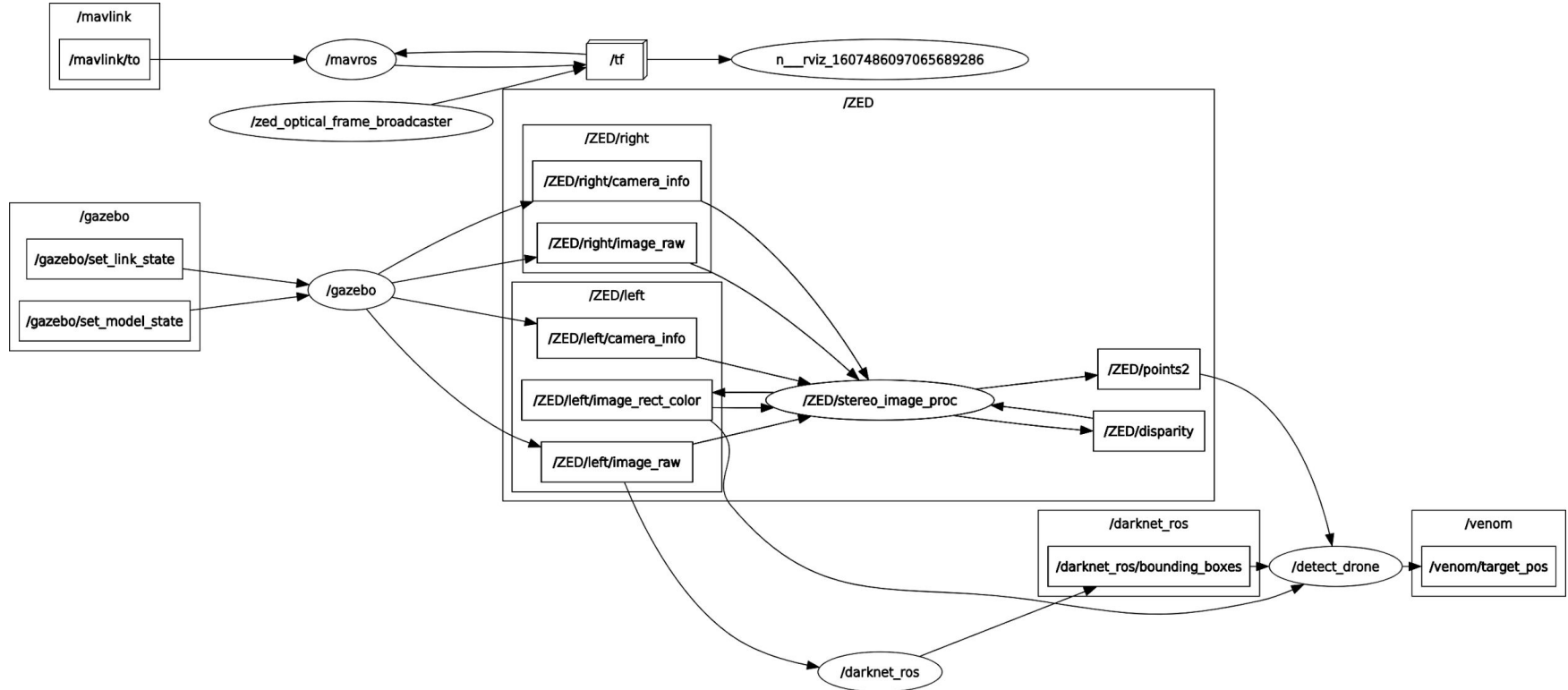
ROS Nodes:

- **detect_cloud:** Centroid estimation in 3D
- **darknet_ros:** ROS wrapper for Darknet based Tiny YOLOv3
- **stereo_image_proc:** Point Cloud Reconstruction
- **camera_tf:** Static Transform for Camera Angle
- **Mavros:** Ardupilot MAVlink interaction
- **Mavproxy GCS:** Ground Control Station
- **Gazebo:** 3D Simulation Environment

Gazebo Simulation

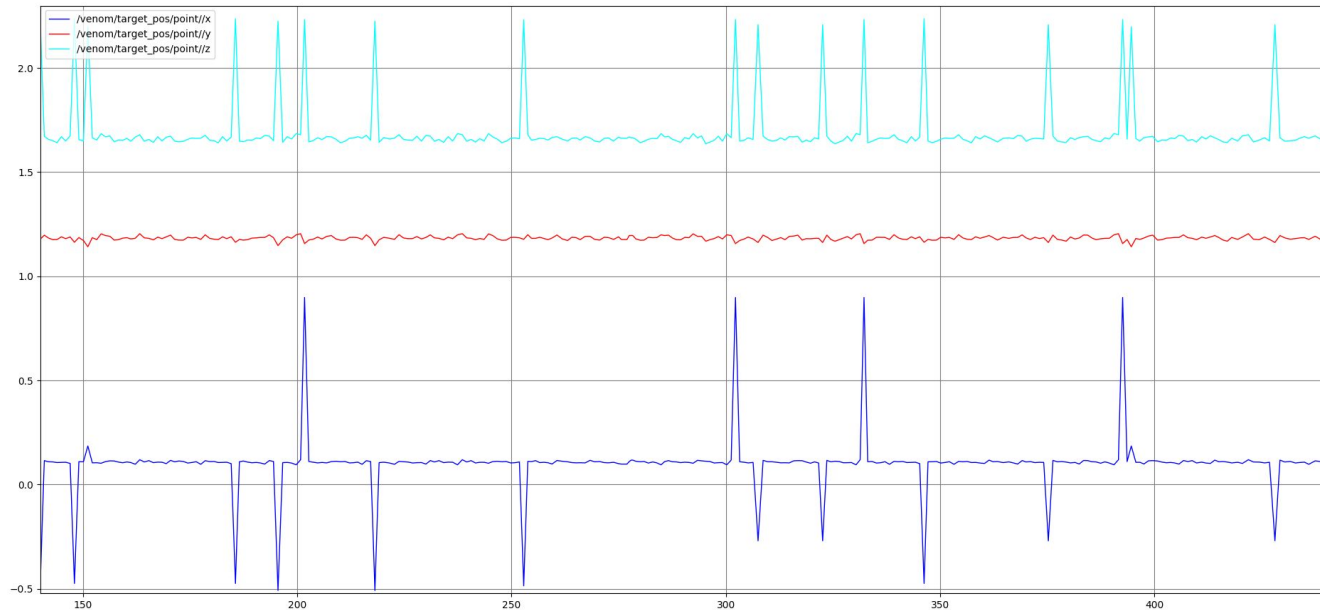


Integration: ROS

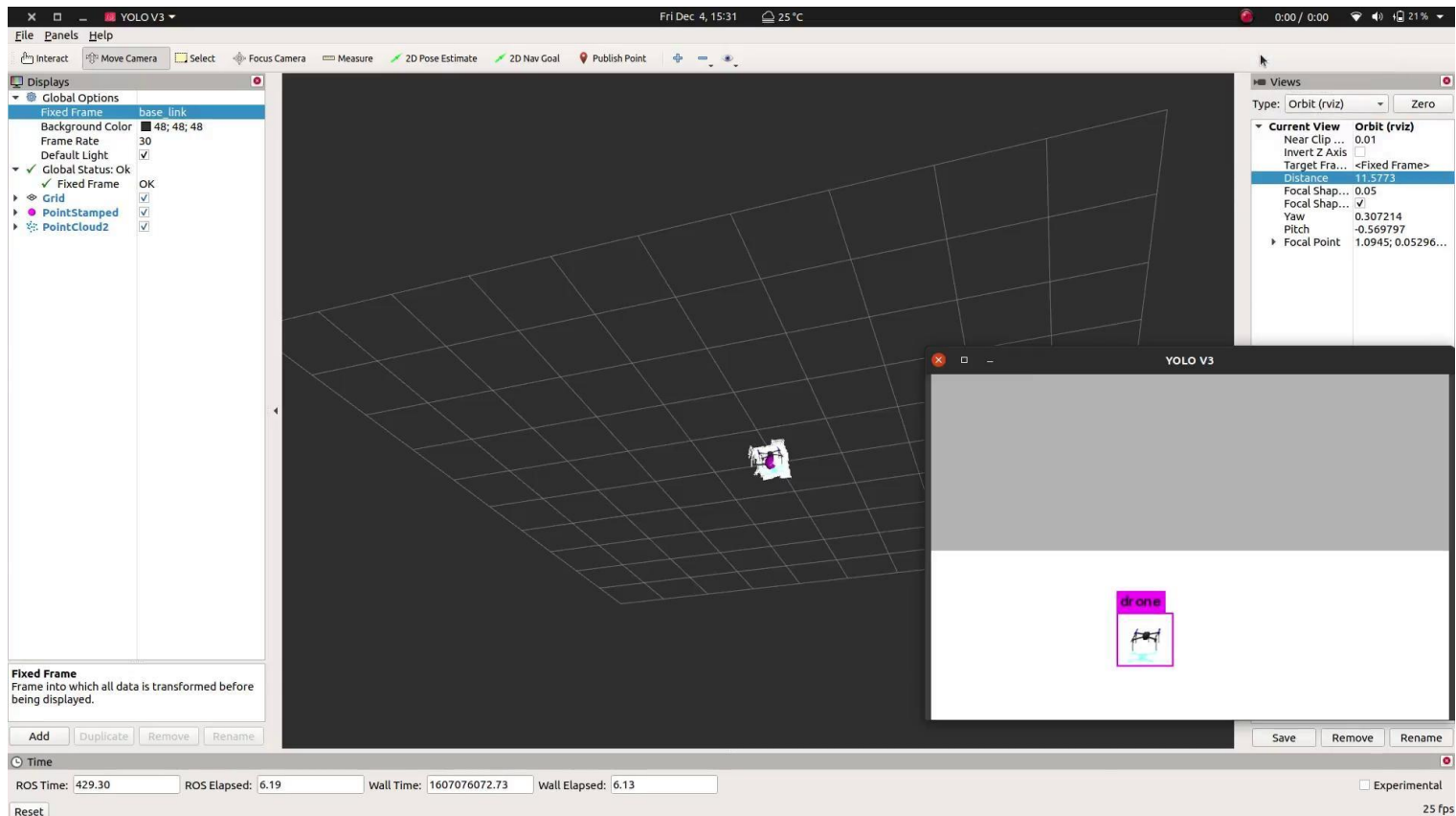


Performance Benchmarking

- Accuracy of 2D Drone Detection: **77% on 0.5 IoU**
- Performance on Nvidia GeForce 940MX: **20 FPS**
- Detection Range: Upto 10 meters in Gazebo Environment
- 3D Detection accuracy in hover mode:



Simulations Results



Limitations faced & Possible Optimizations

- **Detection Model:**
 - Model Optimization: SSD, YOLOv5
 - Lack of labeled stereo-camera based drone dataset
 - Use of Nvidia Jetson TX2
 - 2D drone detection in grayscale
- **Simulation Environment:**
 - ZED SDK was unavailable for Gazebo
 - Camera Calibration was not optimum in for custom made stereo-camera
 - Optimizing the statistical outlier removal
- **Hardware Acceleration**
 - Intel RealSense: Better point clouds while offloading the Point Cloud generation
 - Use of GPU/FPGAs for Visual Inertial Odometry and SLAM

Future Scope

- Performing optimisations in current codebase
- Synthetic dataset creation and training 3D detection models
- Onboard Deployment of 3D drone detection
- Visual Tracking and its onboard deployment

Thank You!
Questions?